

microGPT — Lab 1 Cevapları

Tüm Aıştırmaların Cevap Anahtarı

Bu dosya yalnızca eğitimci / kendi kendine değerlendirme içindir

Bu dosya Lab 1'deki üç alıştırmaların tüm soru cevaplarını içermektedir. Her cevap soru ile birlikte gösterilmiştir.

Aıştırma 1: Token Analizi — Cevaplar

S1: Programı çalıştırdığınızda kaç farklı token bulunmaktadır? BOS dahil vocab_size nedir?

Cevap:

isimler.txt dosyasındaki tüm karakterler büyük harf ve Türkçe özel karakterlerden oluşur.

Benzersiz karakter sayısı: 30

' ' (boşluk) A B C D E F G H I J K L M N O P R S T U V Y Z Ç Ö Ü Ğ İ Ş

BOS özel tokeni: +1

Toplam vocab_size = 31

S2: Türkçe isimlerde hiç kullanılmayan Latin harfleri hangileridir? Neden vocabulary'de yoklar?

Cevap:

Türkçe alfabesinde bulunmayan harfler: Q, W, X

Tokenizer yalnızca eğitim verisinde gördüğü karakterleri vocabulary'e ekler.

isimler.txt'deki isimlerin tamamı Türkçe olduğundan Q/W/X içeren hiçbir isim yoktur.

Dolayısıyla bu karakterler uchars listesine girmez ve token ID almazlar.

S3: isimler.txt dosyasında büyük harfli isimler ile birlikte küçük harfli isimler kullansaydık vocab_size nasıl değişirdi?

Cevap:

Tokenizer yalnızca eğitim verisinde gördüğü karakterleri vocabulary'e ekler.

Büyük ve küçük harf farklı Unicode kod noktaları olduğundan ayrı token ID'si alır.

Örnek: 'A' (ID=1) ve 'a' (ID=31) birbirinden bağımsız tokendir.

Mevcut durum (yalnızca büyük harf):

30 benzersiz karakter + 1 BOS = vocab_size: 31

Her ismin hem büyük hem küçük harfli biçimi bulunsaydı:

Büyük harf karakterleri : 30 (boşluk + A–Z + Ç Ö Ü Ğ İ Ş)

Küçük harf eklentisi : +29 (boşluk zaten ortak; yeni: a–z + ç ö ü ğ ı ş)

Toplam benzersiz karakter: 30 + 29 = 59

vocab_size = 59 + 1 BOS = 60

Sonuç: vocab_size yaklaşık 2 katına çıkar ($31 \rightarrow 60$).

Bu durum wte ve lm_head matrislerini büyüterek parametre sayısını artırır ve aynı kalıpları öğrenmek için daha fazla eğitim adımı gerektirir.

S4: BOS (Beginning of Sequence) tokeninin görevi nedir? Modelin eğitiminde ne işe yarar?

Cevap:

BOS tokeni iki işlev üstlenir:

1. Başlangıç sinyali: Çıkarım sırasında modele 'yeni bir isim üret' komutunu verir.
Model ilk girdi olarak BOS'u alır ve ardından karakterleri tahmin eder.
2. Bitiş sinyali: Model bir ismin sonunda BOS üretince döngü durur.
Böylece modelin ne zaman duracağını öğrenmesi için ayrı bir EOS tokenine gerek kalmaz.

Eğitimde her isim [BOS, k1, k2, ..., kn, BOS] şeklinde çerçevelenir;
model son BOS'u da tahmin etmeyi öğrenir.

Alıştırma 2: Modüler Yapı — Cevaplar

S1: train.py ve test.py ayrımının gerçek dünya ML projelerinde sağladığı avantajlar nelerdir?

Cevap:

- a) Zaman tasarrufu: Modeli bir kez eğitip defalarca test edebilirsiniz;
her test için sıfırdan eğitmek gerekmez.
- b) Depolama: Eğitilmiş model kaydedilerek başka ortamlarda (sunucu, bulut) kullanılabilir.
- c) Tekrarlanabilirlik: Aynı model.pkl ile her test çalıştırması tutarlı sonuç verir.
- d) İş bölümü: Veri bilimcisi eğitimi, mühendis servis katmanını ayrı geliştirebilir.
- e) Hata ayıklama: Eğitim ve çıkarım sorunları izole biçimde incelenebilir.

S2: train.py çalıştırdıktan sonra model.pkl dosyasının boyutu nedir? Parametre sayısı ile ilişkisi?

Cevap:

model.pkl boyutu: yaklaşık 40 KB (sisteme ve pickle protokolüne göre değişir).

Model parametre sayısı: 4 320 (vocab_size=31 ile)

wte=496, wpe=256, lm_head=496, attn=1024, mlp=2048

pickle her Python float'ı ~9 byte ile kodlar (1 opcode + 8 byte veri).

4 320 params × 9 byte ≈ 39 KB + dict/list/string overhead ≈ 1–2 KB

Toplam ≈ 40–42 KB

Not: Orijinal microgpt (vocab_size=27, sadece a-z + BOS) ile daha az parametre → daha küçük dosya.

Numpy/safetensors formatları daha verimli serileştirme sağlar.

S3: Modelin kaç parametresi bulunmaktadır?

Cevap:

microgpt.py'deki varsayılan hiperparametrelerle:

n_embd=16, vocab_size=31, block_size=16, n_head=4, n_layer=1

wte : $31 \times 16 = 496$

wpe : $16 \times 16 = 256$

lm_head : $31 \times 16 = 496$

attn_wq/wk/wv/wo : $4 \times (16 \times 16) = 1\,024$

mlp_fc1 : $64 \times 16 = 1\,024$ ($4 \times n_embd \times n_embd$)

mlp_fc2 : $16 \times 64 = 1\,024$ ($n_embd \times 4 \times n_embd$)

TOPLAM : $496 + 256 + 496 + 1024 + 1024 + 1024 = 4\,320$

Not: isimler.txt vocab_size=31 olduğundan orijinal kodun (vocab_size=27) çıktısından farklıdır.

S4: random.seed(42) olmadan test.py'yi iki kez çalıştırsaydınız aynı isimleri üretir miydi?

Cevap:

Hayır, her çalıştırmada farklı isimler üretilirdi.

random.choices() sistem saatine dayalı rastgele bir tohum kullanır.

random.seed(42) ile tohum sabitlendiğinde rastgele sayı dizisi deterministik olur

ve her çalıştırmada birebir aynı örnekleme yapılır.

Model ağırlıkları (model.pkl) değişmediğinden farklılık yalnızca örnekleme rastgeleliğinden kaynaklanır — model logitleri her zaman aynıdır.

Alıştırma 3: Temperature Parametresi — Cevaplar

Doldurulmuş Örnek Tablo (yaklaşık değerler — modele göre değişir)

Temperature	Örnek 1	Örnek 2	Örnek 3	Gözlem
0.1	MEHMET	MEHMET	MEHMET	Tekrarlayan
0.5	EMRE	GÜLŞAH	ALİ	Dengeli
1.0	SERKAN	MÜJDE	HAYRÜYE	Çeşitli
2.0	ÇTMBÖ	AŞZĞÜR	İÖBŞR	Anlamsız

S1: T=0.1 ile T=2.0 arasında üretilen isimler arasındaki en belirgin fark nedir?

Cevap:

T=0.1: Model her seferinde aynı ya da çok benzer isimleri üretir. Dağılım tek bir token üzerinde yoğunlaşır; çeşitlilik neredeyse sıfırdır.

T=2.0: Olasılık dağılımı düzleşir, model neredeyse rastgele karakter seçer. Üretilen diziler anlamsız harf kombinasyonlarına dönüşür.

Fark: T=0.1'de aşırı belirlilik (ezber/kalıp), T=2.0'de aşırı rastgelelik (gürültü).

S2: Hangi temperature değerinde üretilen isimler gerçek Türkçe isimlere en çok benziyor?

Cevap:

Genellikle T=0.5 en iyi dengeyi sağlar (orijinal kodun varsayılanı da budur). Bu değerde model, yüksek olasılıklı (öğrenilmiş) kalıpları ön plana çıkarırken tamamen deterministik olmayan çeşitli isimler üretir.

T=1.0 da kabul edilebilir sonuçlar verebilir; bazı modellerde daha doğal görünür.

Neden: Düşük temperature, modelin en 'güvenli' tahminlerine kilitleyerek eğitim verisindeki en sık isimleri tekrarlar. Orta temperature ise öğrenilen fonotaktik (ses dizilimi) örüntüleri koruyarak yeni kombinasyonlar üretir.

S3: random.seed(42) ile aynı temperature için her çalıştırmada aynı sonuçlar mı üretilir?

Cevap:

Evet, random.seed(42) tohumu sabitlendiğinde rastgele sayı üretici aynı diziye üretir. Bu nedenle aynı temperature değeriyle her çalıştırmada birebir aynı isimler elde edilir.

Tohumu kaldırırsanız (veya farklı bir değer verirsiniz) sonuçlar değişir.

Bu özellik sonuçların tekrarlanabilir (reproducible) olmasını sağlar; akademik çalışmalarda ve hata ayıklamada büyük önem taşır.

S4: Temperature parametresi ile model ağırlıkları arasındaki temel fark nedir? Hangisi eğitimden etkilenir?

Cevap:

Model ağırlıkları (parametreler): Eğitim sırasında Adam optimizörü tarafından güncellenir. Veriyi 'öğrenen' kısımdır; model.pkl'a kaydedilir.

Temperature: Bir çıkarım (inference) hiperparametresidir. Eğitim sırasında kullanılmaz; yalnızca örnekleme adımında softmax çıktısını ölçekler.

Kısaca: Ağırlıklar eğitimden etkilenir, temperature etkilenmez.

Temperature'ı değiştirmek için modeli yeniden eğitmek gerekmez.

S5: Veri seti genişletme (data augmentation) amacıyla isim üretecek olsanız hangi temperature değerini seçerdiniz?

Cevap:

Önerilen aralık: $T=0.7$ – $T=1.0$

Gerekçe:

- T çok düşük (≤ 0.3): Model eğitim verisindeki isimleri tekrar eder, augmentation amacı ortadan kalkar.
- T çok yüksek (≥ 1.5): Üretilen isimler dilbilgisel açıdan geçersiz hale gelir, gerçekçi isim kalıplarından uzaklaşır.
- $T \approx 0.8$: Model öğrendiği fonotaktik kuralları korurken yeterli çeşitlilik sağlar — hem özgün hem de Türkçe isim kalıplarına uyan veriler üretir.